챕터 2. 동기식 MSA 구현 - 서비스를 연결하다

오픈이 : "선배님, 그럼 어떤 것부터 시작하면 될까요?"

선배 : "우선 회원, 주문, 상품, 배달, 이렇게 기능별로 서비스를 나누는 거예요. 그리고 서비스끼리 전화를 거는 것처럼 직접 호출하는 거죠."

방향이 정해졌으니 이제 만들어 보겠습니다.

이번 챕터의 핵심은 주문 서비스입니다. 주문 서비스에서 주문 요청이 들어오면, 주문 서비스가 상품 서비스와 배달 서비스를 직접 호출하고 응답하는 흐름을 따라가 보겠습니다.

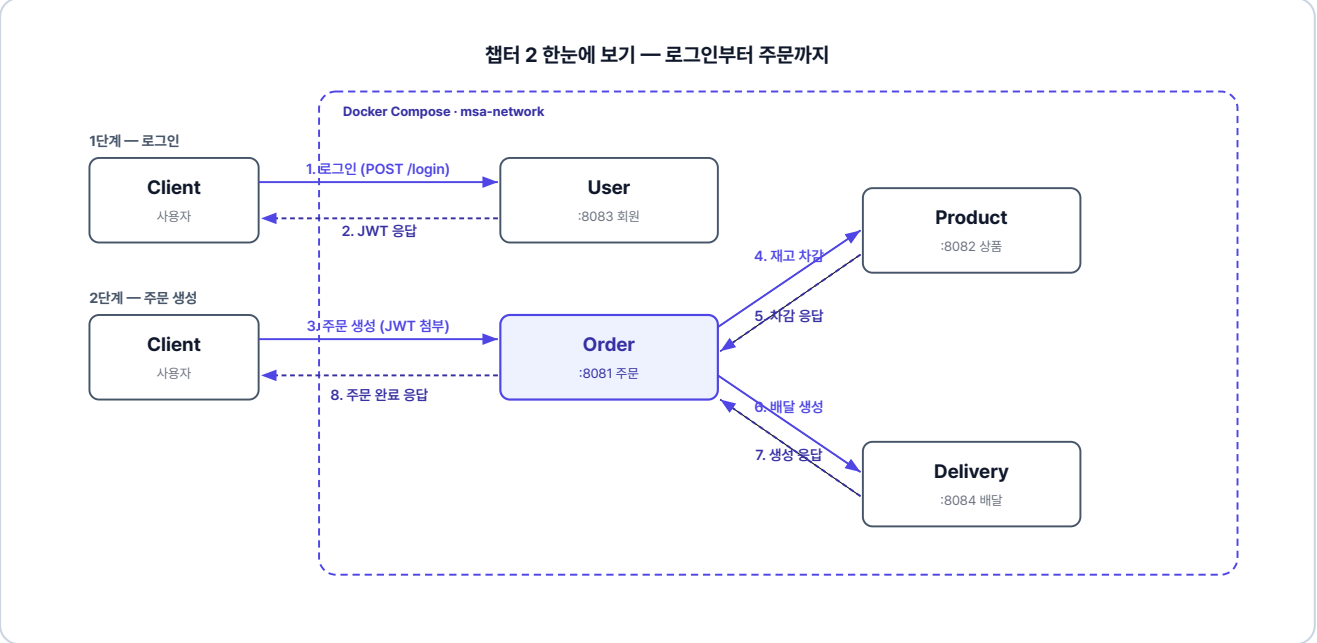


그림 2-1. 챕터 2 한눈에 보기 - 로그인부터 주문까지

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex01
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex01` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex01 디렉토리

```
ex01/
├── delivery/           # 포트 8084
├── docker-compose.yml  # 전체 서비스 실행
├── order/              # 포트 8081
├── product/            # 포트 8082
└── user/               # 포트 8083
```

각 서비스 내부는 동일한 구조입니다. 주문 서비스 기준으로 보여드리며, 회원/상품/배달 서비스도 같은 구조입니다.

주문 서비스 패키지 구조

```
src/main/java/com/metacoding/order/
├── adapter/                                # 주문 서비스에만 존재
│   ├── DeliveryClient.java                 # [참고] 배달 서비스 호출
│   ├── dto/                                # 어댑터용 DTO
│   └── ProductClient.java                  # [참고] 상품 서비스 호출
├── core/
│   ├── config/
│   │   ├── RestClientConfig.java           # [참고] JWT 헤더 전달 인터셉터
│   │   └── WebConfig.java                  # [참고] JWT 필터 등록
│   ├── filter/
│   │   └── JwtAuthenticationFilter.java    # [참고] JWT 인가 필터
│   ├── handler/
│   │   ├── ex/                             # 커스텀 예외 (Exception400~500)
│   │   └── GlobalExceptionHandler.java     # [참고] 전역 예외 처리
│   └── util/
│       ├── JwtProvider.java                # [참고] JWT 파싱/검증
│       ├── JwtUtil.java                    # [참고] JWT 생성
│       └── Resp.java                       # [참고] 표준 응답 래퍼
└── orders/
    ├── Order.java                          # [참고] JPA 엔티티
    ├── OrderController.java                # [참고] REST 컨트롤러
    ├── OrderRepository.java                # [참고] Spring Data JPA
    ├── OrderRequest.java / OrderResponse.java  # [참고]
    ├── OrderService.java                   # [작성] 비즈니스 로직
    └── OrderStatus.java                    # [참고] 주문 상태 enum
Dockerfile                                  # [참고] Docker 이미지 빌드
```

> **회원/상품/배달 서비스는 `adapter/` 패키지와 `RestClientConfig`가 없고, 나머지 구조는 동일합니다.**

3. 실습 환경

도구	용도	비고
Docker Desktop	4개 서비스를 컨테이너로 실행	https://www.docker.com/products/docker-desktop/
Hoppscotch	API 호출 결과 확인	https://hoppscotch.io/ (설치 불필요, 브라우저 확장만 추가)

4. 실습 순서

1. 공통 설정(JWT·표준 응답·예외 처리)이 담긴 `core/` 패키지 살펴보기
2. 회원·상품·배달 서비스의 핵심 코드 살펴보기
3. 주문 서비스에 RestClient + 보상 트랜잭션 작성하기
4. Docker Compose로 4개 서비스를 한 번에 띄우고 시나리오 3개 검증하기

2.1 공통 설정 - 모든 서비스가 공유하는 뼈대

각 서비스는 **Spring Boot**로 만들어진 독립 프로젝트입니다. 서버가 분리되어 **세션을 공유할 수 없으므로 JWT 토큰**으로 인증합니다.

4개 서비스가 공통으로 쓰는 **JWT 인증·표준 응답·예외 처리**는 `core/` 패키지에 모아 둡니다. 각 컴포넌트의 역할은 다음과 같습니다.

컴포넌트	역할
JwtAuthenticationFilter	매 요청마다 JWT를 검증하고 사용자 정보를 컨트롤러로 전달합니다.
JwtUtil / JwtProvider	JwtUtil은 토큰을 발급하고 검증하는 핵심 로직이고, JwtProvider는 요청 헤더에서 토큰을 꺼내 JwtUtil에 넘깁니다.
Resp	모든 API 응답을 동일한 형태로 통일하는 래퍼입니다.

컴포넌트	역할
GlobalExceptionHandler	전역에서 발생한 예외를 잡아 일관된 에러 응답으로 변환합니다.
WebConfig	JWT 필터를 인증이 필요한 경로에 등록합니다.

주문 서비스로 가기 전, 회원·상품·배달 서비스는 참고 코드라 클래스 단위로 간단히 살펴봅니다. 자세한 구현은 완성 레포(GitHub)를 참고하세요.

2.2 회원 서비스 - JWT로 로그인하다

회원 서비스는 **로그인과 사용자 조회**를 담당합니다. 사용자가 `POST /login`으로 아이디와 비밀번호를 보내면, 회원 서비스가 DB에서 조회하고 비밀번호를 검증합니다. 검증에 성공하면 **JWT 토큰**을 응답 데이터에 담아 돌려줍니다. 이 토큰이 이후 모든 서비스 요청의 **인증 수단**이 됩니다.

HTTP 메서드	경로	기능
POST	/login	로그인 (JWT 발급)
GET	/api/users/{userId}	사용자 조회

2.2.1 클래스 구성

회원 서비스의 로그인과 사용자 조회를 다음 다섯 클래스가 처리합니다.

클래스	역할
User	사용자 정보를 담는 엔티티입니다.
UserRepository	사용자 데이터를 DB에서 조회·저장합니다.
UserRequest / UserResponse	회원 API의 요청과 응답 형식을 정의합니다.
UserController	로그인과 사용자 조회 API를 제공합니다.
UserService	로그인 시 비밀번호를 검증하고 JWT를 발급하며, 사용자 조회 요청을 처리합니다.

2.3 상품 서비스 - 재고를 관리하다

상품 서비스는 **상품 목록 조회와 재고 증감**을 담당합니다. 주문 서비스가 주문을 생성할 때 **재고 감소 API**를 호출하고, 주문이 취소되거나 실패하면 **재고 증가 API**로 되돌립니다.

HTTP 메서드	경로	기능
GET	/api/products/{productId}	상품 조회
PUT	/api/products/{productId}/decrease	재고 감소
PUT	/api/products/{productId}/increase	재고 증가

2.3.1 클래스 구성

상품 조회와 재고 증감을 다음 다섯 클래스가 처리합니다.

클래스	역할
Product	상품 정보를 담는 엔티티이며, 재고 증감 로직을 자기 안에 둡니다.
ProductRepository	상품 데이터를 DB에서 조회·저장합니다.
ProductRequest / ProductResponse	상품 API의 요청과 응답 형식을 정의합니다.
ProductController	상품 조회와 재고 증감 API를 제공합니다.
ProductService	재고 감소 전 상품 존재·재고·가격을 검증한 뒤 재고를 줄이거나 늘립니다.

2.4 배달 서비스 - 배달을 생성하고 취소하다

배달 서비스는 **배달 생성과 취소**를 담당합니다. 회원이나 상품 서비스와 달리, 주문과 배달에는 현재 진행 상황을 나타내는 **상태(`status`) 값**이 있습니다. 대기 상태인 `PENDING`으로 시작해 처리가 완료되면 `COMPLETED`, 취소되면 `CANCELLED`로 바뀝니다.

HTTP 메서드	경로	기능
POST	/api/deliveries	배달 생성
GET	/api/deliveries/{deliveryId}	배달 조회
PUT	/api/deliveries/{orderId}	배달 취소

2.4.1 클래스 구성

배달 생성과 취소를 다음 여섯 클래스가 처리합니다.

클래스	역할
Delivery	배달 정보와 현재 상태(PENDING / COMPLETED / CANCELLED)를 담는 엔티티입니다.
DeliveryStatus	배달 상태(대기·완료·취소)를 정의하는 열거형입니다.
DeliveryRepository	배달 데이터를 DB에서 조회·저장합니다.
DeliveryRequest / DeliveryResponse	배달 API의 요청과 응답 형식을 정의합니다.
DeliveryController	배달 생성·조회·취소 API를 제공합니다.
DeliveryService	배달 생성 시 완료까지 처리하며, 조회와 취소도 처리합니다.

2.5 주문 서비스 - 보상 트랜잭션의 현장

주문 서비스는 **주문 생성·조회·취소**를 담당합니다. 주문 요청 처리를 위해 상품·배달 서비스를 직접 호출하는 흐름의 중심에 있습니다.

HTTP 메서드	경로	기능
POST	/api/orders	주문 생성
GET	/api/orders/{orderId}	주문 조회

HTTP 메서드	경로	기능
PUT	/api/orders/{orderId}	주문 취소

2.5.1 클래스 구성

주문 생성과 보상 트랜잭션을 다음 여섯 클래스가 처리합니다.

클래스	역할
Order	주문 정보와 현재 상태(PENDING / COMPLETED / CANCELLED)를 담는 엔티티입니다.
OrderStatus	주문 상태(대기·완료·취소)를 정의하는 열거형입니다.
OrderRepository	주문 데이터를 DB에서 조회·저장합니다.
OrderRequest / OrderResponse	주문 API의 요청과 응답 형식을 정의합니다.
OrderController	주문 생성·조회·취소 API를 제공합니다.
OrderService	**[작성]** 주문 생성 시 보상 트랜잭션을 수행하며, 조회와 취소도 처리합니다.

2.5.2 보상 트랜잭션 흐름

서비스의 구조를 파악했으니, 이제 주문 서비스의 진짜 역할을 살펴봅니다. 주문 요청이 들어오면 주문 서비스는 상품·배달 서비스를 호출합니다. 이때 작업이 실패하면, **주문 서비스가 직접 보상 트랜잭션을 실행**해 원래 상태로 되돌립니다.

어느 단계에서 실패하면 무엇을 되돌려야 하는지 먼저 그려 보겠습니다.

단계	동작	실패 시 보상
1	재고 감소 (상품 서비스)	없음 (아직 진행 안 함)
2	배달 생성 (배달 서비스)	재고 복구 (1단계 되돌리기)
3	주문 완료	배달 취소 + 재고 복구 (2, 1단계 되돌리기)

단계	동작	실패 시 보상
4	성공 응답 반환	—

예를 들어 2단계 배달 생성에서 실패하면, 이미 줄어든 재고 한 단계만 되돌리면 됩니다. 3단계에서 실패하면 배달 취소와 재고 복구를 모두 되돌립니다. 진행한 만큼만 역순으로 되돌리려면, **어디까지 갔는지를 코드에서 기록**해 둬야 합니다.

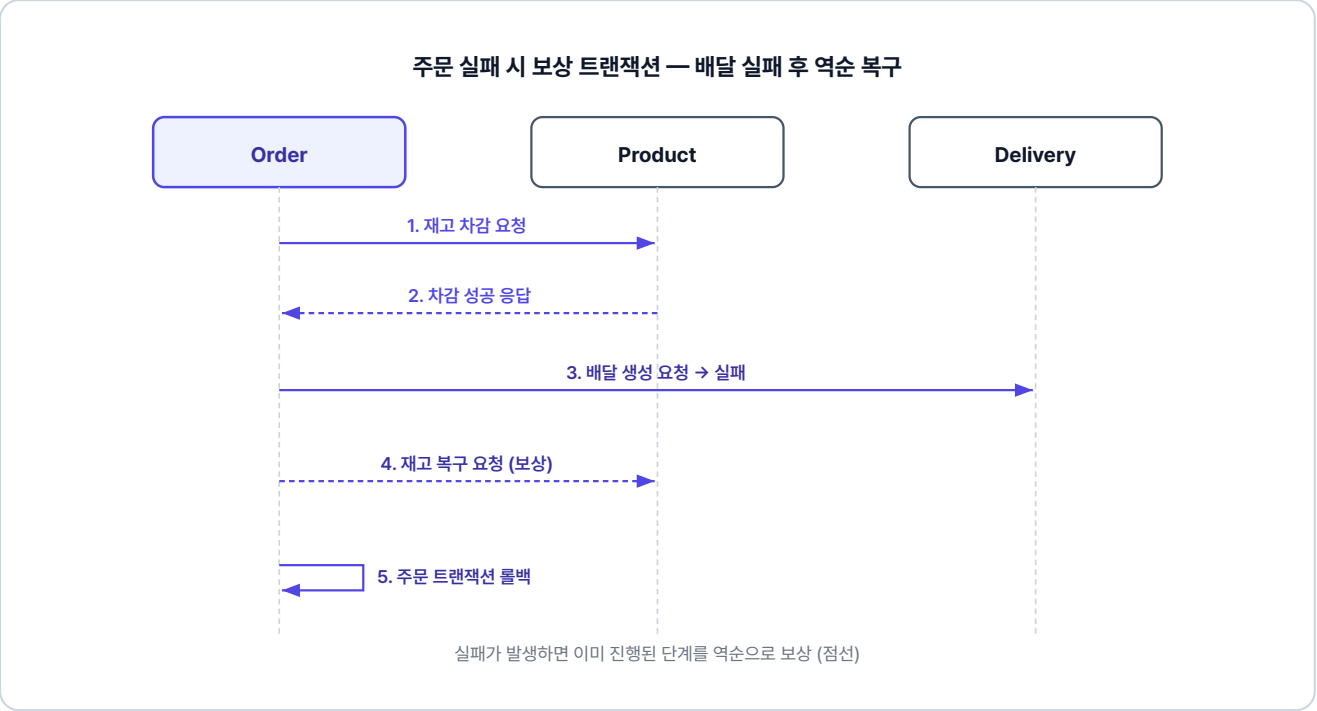


그림 2-2. 주문 실패 시 보상 트랜잭션 흐름

2.5.3 adapter - 외부 서비스 호출 설정

`adapter/` 폴더에는 외부 서비스를 호출하는 **클라이언트** 두 개가 있습니다.

RestClientConfig가 모든 외부 호출에 **JWT 토큰을 자동으로 실어** 줍니다. 그리고 **ProductClient**와 **DeliveryClient**가 각자 상품 서비스와 배달 서비스로 요청을 보냅니다.

클래스	역할
RestClientConfig	RestClient에 인터셉터를 등록하여 외부 호출 시 JWT를 자동 전달합니다.
ProductClient	상품 서비스의 decreaseQuantity(재고 감소)와 increaseQuantity(재고 복구)를 호출합니다.

클래스	역할
DeliveryClient	배달 서비스의 createDelivery(배달 생성)와 cancelDelivery(배달 취소)를 호출합니다.

2.5.4 OrderService - 보상 트랜잭션의 핵심

이제 이번 챕터의 핵심입니다. **보상 트랜잭션 패턴**을 직접 구현합니다. 코드를 읽을 때 `productDecreased`와 `deliveryCreated` **두 플래그를 추적**하면서 읽어보세요. 단계가 성공할 때마다 플래그를 `true`로 바꾸고, 실패 시 catch 블록에서는 `true`**로 표시된 단계만 역순으로 되돌립니다**.

주문 서비스의 `orders/OrderService.java`를 열고 아래 메서드를 작성합니다.

실습 1 **주문 서비스 - orders/OrderService.java. createOrder - 보상 트랜잭션 핵심**

```
@Transactional
public OrderResponse createOrder(int userId, int productId,
        int quantity, Long price, String address) {
    // 보상트랜잭션을 위한 변수 선언
    boolean productDecreased = false;
    boolean deliveryCreated = false;

    // 보상트랜잭션에서 id를 전달해야해서 상위로 빼둠
    Order createdOrder = null;

    try {
        // 1. 주문 생성
        createdOrder = orderRepository.save(
                Order.create(userId, productId, quantity, price));

        // 최소 주문 금액 검증
        if (quantity * price < 1000) {
            throw new Exception400("최소 주문 금액은 1,000원입니다.");
        }

        // 2. 상품 재고 차감
        productClient.decreaseQuantity(
                new ProductRequest(productId, quantity, price));
        productDecreased = true;

        // 3. 배달 생성
        deliveryClient.createDelivery(
                new DeliveryRequest(createdOrder.getId(), address));
        deliveryCreated = true;
```

```
            // 4. 주문 완료
            createdOrder.complete();
            return OrderResponse.from(createdOrder);

        } catch (Exception e) {
            // 배달 취소
            if (deliveryCreated) {
                deliveryClient.cancelDelivery(createdOrder.getId());
            }

            // 재고 복구
            if (productDecreased) {
                productClient.increaseQuantity(
                        new ProductRequest(productId, quantity, price));
            }
            throw new Exception500("주문 생성 중 오류가 발생했습니다: " + e.getMessage());
        }
    }
```

주문 데이터는 catch에서 따로 되돌리지 않아도 자동으로 롤백됩니다. 외부 서비스(상품·배달)만 직접 보상하면 됩니다.

재고를 줄였으면 다시 늘리고, 배달을 만들었으면 취소하고... 그 말이 이거였구나.

> **MSA에서 int/Long 대신 UUID를 쓰는 이유**
>
> 이 책은 주문 ID를 순차 증가 방식(int/Long)으로 만듭니다. 단일 데이터베이스를 쓰는 작은 서비스에서는 이 방식이 단순하고 편합니다. 하지만 서비스 규모가 커지면 임의의 난수인 UUID를 사용해야 합니다. 가장 큰 이유는 **보안**과 **분산 환경에서의 충돌 방지** 때문입니다.
>
> 순차 번호는 1037 다음이 1038임을 누구나 알 수 있습니다. 외부에서 **주소의 번호만 바꿔 악의적으로 남의 주문을 조회하거나, 하루 전체 주문량을 쉽게 유추할 수 있습니다**. 반면 UUID는 랜덤 값이기 때문에 다음 번호나 주문량을 추측할 수 없습니다. 또한, 시스템 규모가 커져 데이터베이스를 여러 대로 나누게 되면, 각 DB가 **동일한 ID를 생성해 번호가 겹치는 치명적인 문제**가 생길 수 있습니다. UUID를 사용하면 데이터를 여러 곳으로 분산시켜도 번호 충돌이 발생하지 않습니다.

2.6 Docker Compose - 네 개의 서비스를 한 번에 실행하다

시나리오를 따라가기 전에, 각 서비스에 어떤 데이터가 미리 등록되어 있는지 살펴봅니다. 이번 챕터는 **H2 in-memory DB**를 사용합니다. 데이터 정의는 각 서비스의 `db/data.sql`에 있습니다.

서비스	더미 데이터
회원	계정 3개: ssar, cos, love (비밀번호는 모두 1234)
상품	MacBook Pro(재고 10), iPhone 15(**재고 0 품절**), AirPods(재고 10)
배달	주문 3건에 대응하는 배달 데이터, 모두 COMPLETED 상태
주문	사용자별 주문 3건(완료·취소·대기)

2.6.1 서비스 실행

각 서비스는 **동일한 구조**의 Dockerfile로 **컨테이너 안에서 빌드하고 실행**됩니다. 주문 서비스의 Dockerfile을 예로 살펴봅니다.

참고 **order/Dockerfile**

```
FROM eclipse-temurin:21-jdk              # JDK 21 베이스 이미지
WORKDIR /app                             # 작업 디렉터리 설정
COPY . .                                 # 프로젝트 소스 복사
RUN chmod +x gradlew                     # gradlew 실행 권한 부여
RUN ./gradlew bootJar -x test            # 테스트 없이 실행 가능한 JAR 빌드
RUN cp build/libs/*.jar app.jar          # 빌드된 JAR를 app.jar로 복사
ENTRYPOINT ["java", "-jar", "app.jar"]   # 컨테이너 시작 시 JAR 실행
```

`ex01` 디렉토리의 `docker-compose.yml`은 네 서비스를 **하나의 네트워크로 묶어 한 번에 실행**합니다.

서비스	build.context	호스트:컨테이너 포트	네트워크
order-service	./order	8081:8081	msa-network
user-service	./user	8083:8083	msa-network
product-service	./product	8082:8082	msa-network
delivery-service	./delivery	8084:8084	msa-network

`msa-network`로 묶여 있기 때문에, 컨테이너끼리는 **서비스 이름**(예: `http://product-service:8082`)으로 통신할 수 있습니다.

프로젝트가 위치한 폴더로 이동 후, 터미널에서 Docker Compose로 4개 서비스를 한 번에 빌드하고 실행합니다.

터미널 **Docker Compose 실행**

```
docker compose up
```

ex01 — docker compose up

```
user-service   | Started UserApplication in 4.231 seconds (port: 8083)
product-service   | Started ProductApplication in 4.512 seconds (port: 8082)
order-service   | Started OrderApplication in 5.103 seconds (port: 8081)
delivery-service   | Started DeliveryApplication in 4.687 seconds (port: 8084)
----------------------------------------------------------------------------
4개 서비스 기동 완료
```

그림 2-3. Docker Compose 실행 결과

2.6.2 Hoppscotch와 인터셉터 설정

서비스를 실행했으니, 이제 API를 호출해 잘 동작하는지 확인해 보겠습니다. 이 책에서는 브라우저에서 API를 호출하는 도구인 **Hoppscotch**(https://hoppscotch.io/)를 사용합니다.

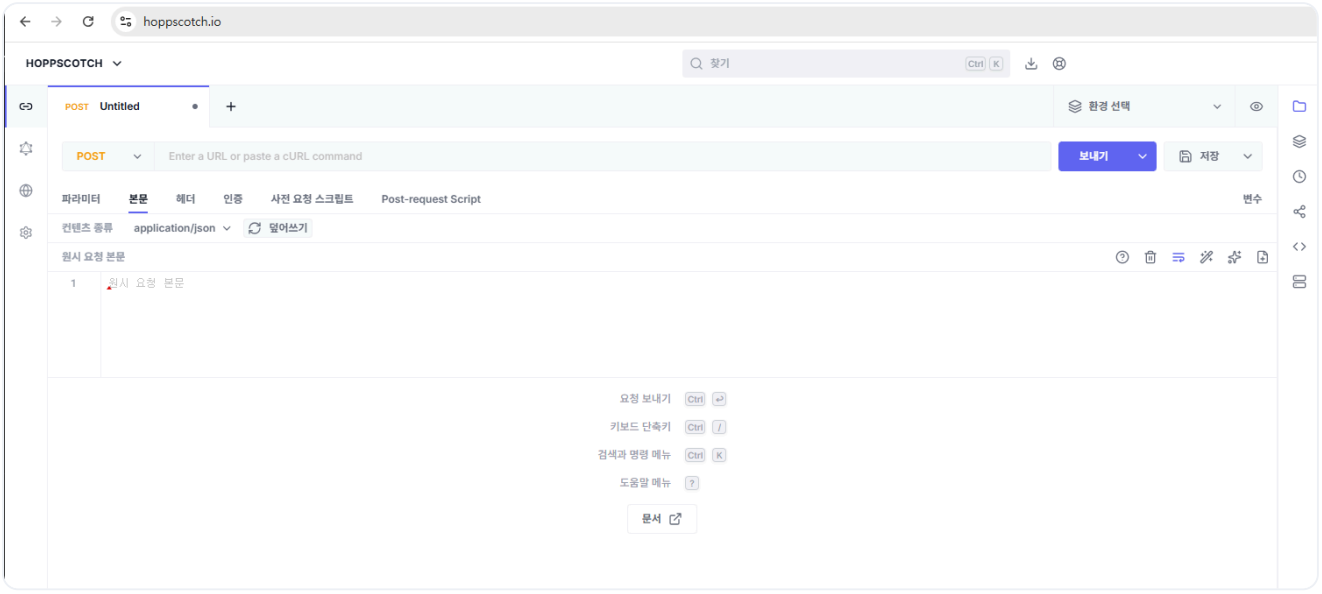


그림 2-4. Hoppscotch 화면

웹 브라우저는 보안 때문에 내 컴퓨터의 localhost로 바로 요청을 보내지 못합니다. 그래서 요청을 대신 전달해 주는 **Hoppscotch Browser Extension**을 Chrome 웹 스토어에서 설치하고, 설정 > Interceptor에서 익스텐션을 선택합니다.

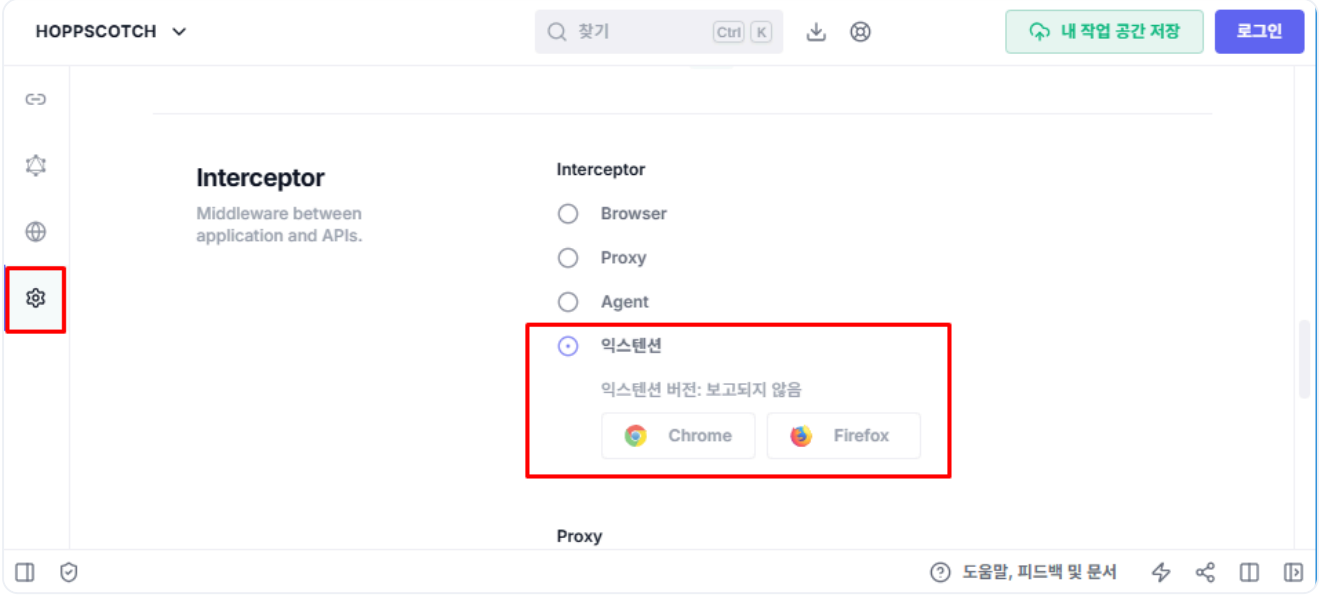


그림 2-5. Browser Extension 인터셉터 설정

2.6.3 시나리오 1: 정상 주문

먼저 로그인하여 JWT 토큰을 받습니다. 이때 콘텐츠 종류(Content-Type)를 `application/json`으로 설정해야 합니다. 이 헤더는 서버에게 "내가 보내는 데이터는 JSON 형식이다"라고 알리는 역할을 합니다.

Hoppscotch 로그인

```
POST http://localhost:8083/login

{
  "username": "ssar",
  "password": "1234"
}
```

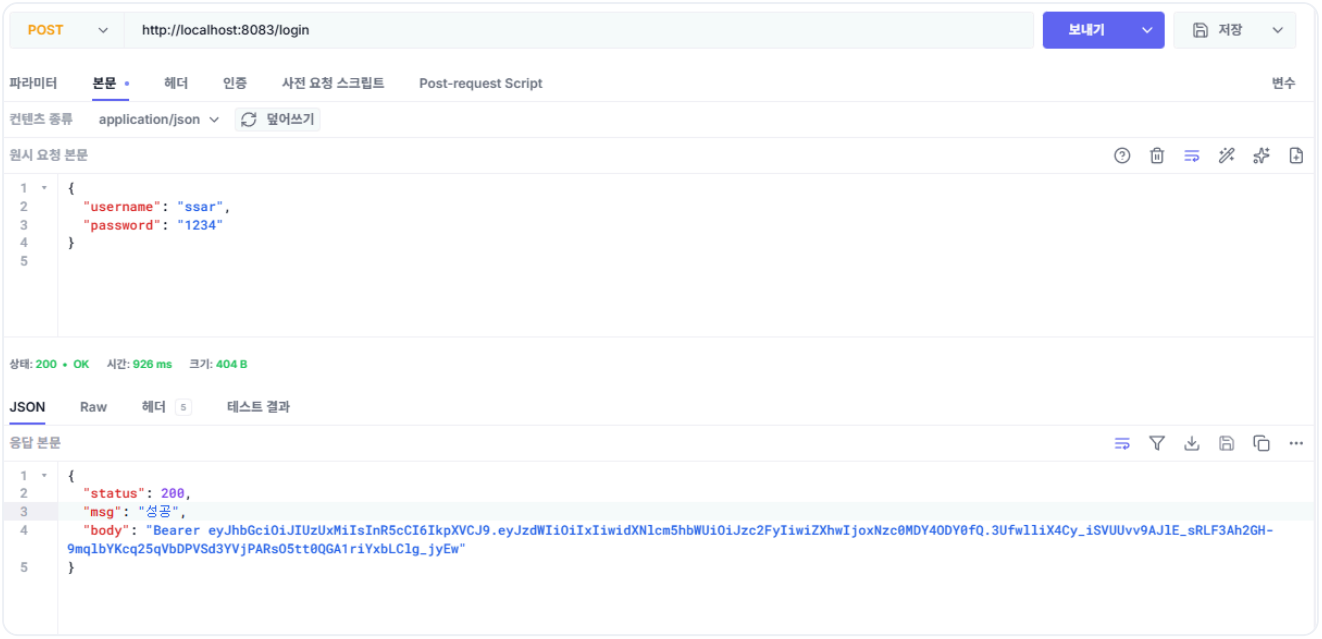


그림 2-6. 로그인 API 호출 결과

응답 데이터에 포함된 JWT 토큰을 확인할 수 있습니다.

받은 토큰을 Hoppscotch의 **인증 > 인증 유형(Bearer)** 항목의 토큰 필드에 넣습니다.

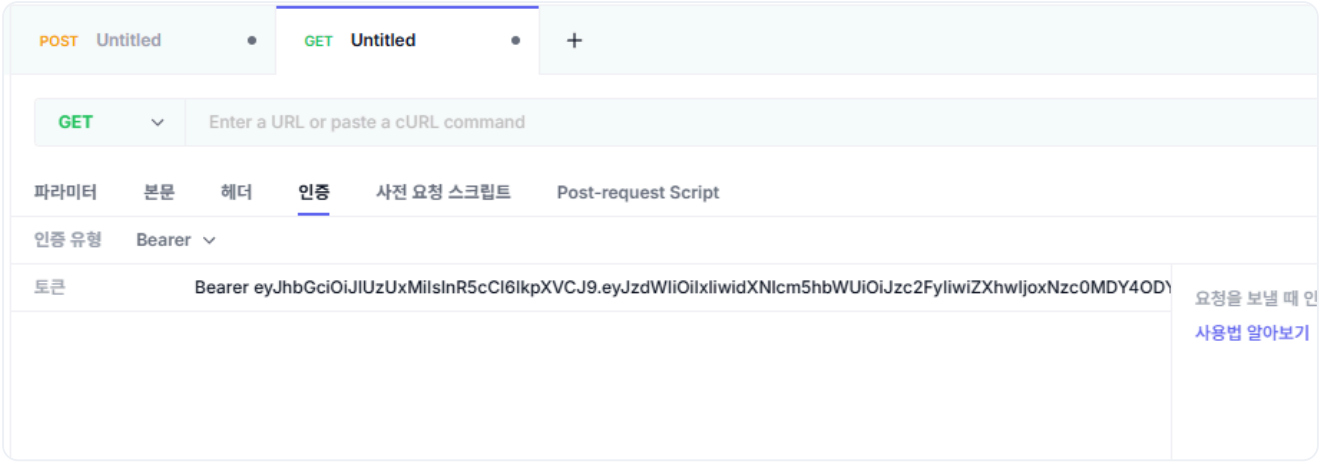


그림 2-7. Bearer 토큰 설정

다음으로 상품 ID가 1인 MacBook Pro 1개를 주문합니다. 요청 데이터는 상품 정보(`productId`, `quantity`, `price`)와 배달 주소(`address`)를 한 번에 담습니다.

Hoppscotch 주문 생성

```
POST http://localhost:8081/api/orders

{
```

```
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

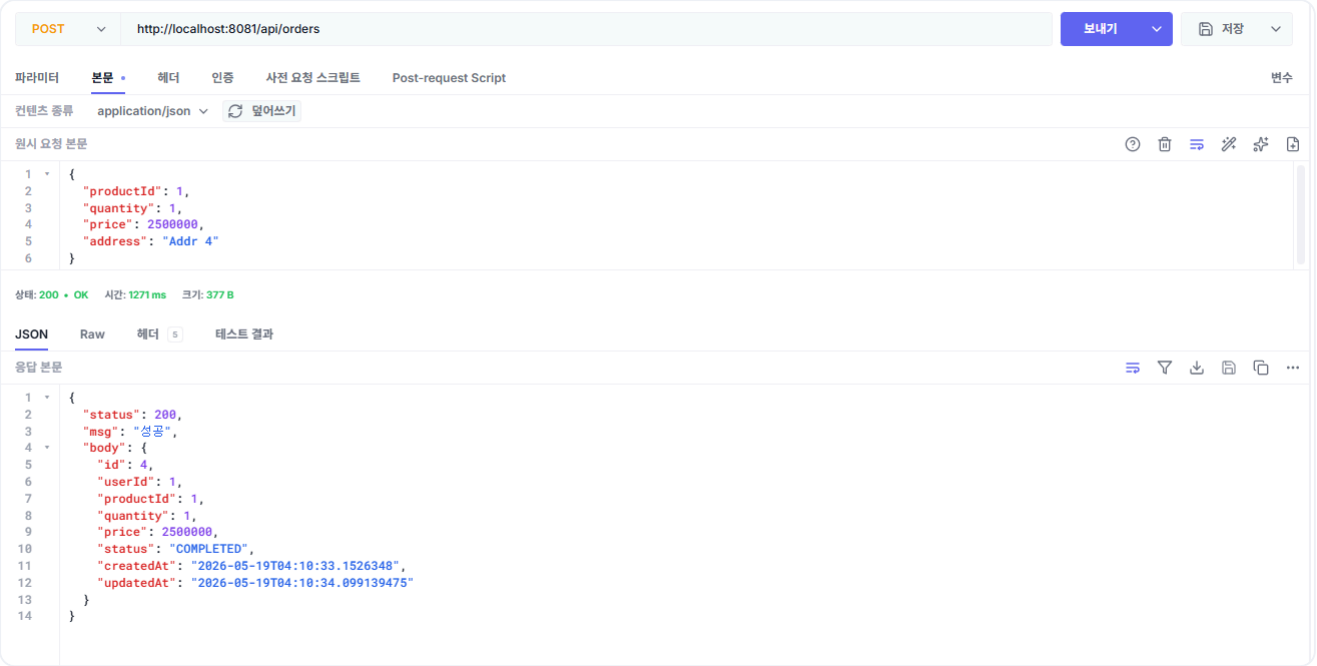


그림 2-8. 주문 생성 API 호출 결과

주문이 성공하면 상품 서비스에서 재고가 10 → 9로 줄어듭니다.

Hoppscotch **재고 조회**

```
GET http://localhost:8082/api/products/1
```

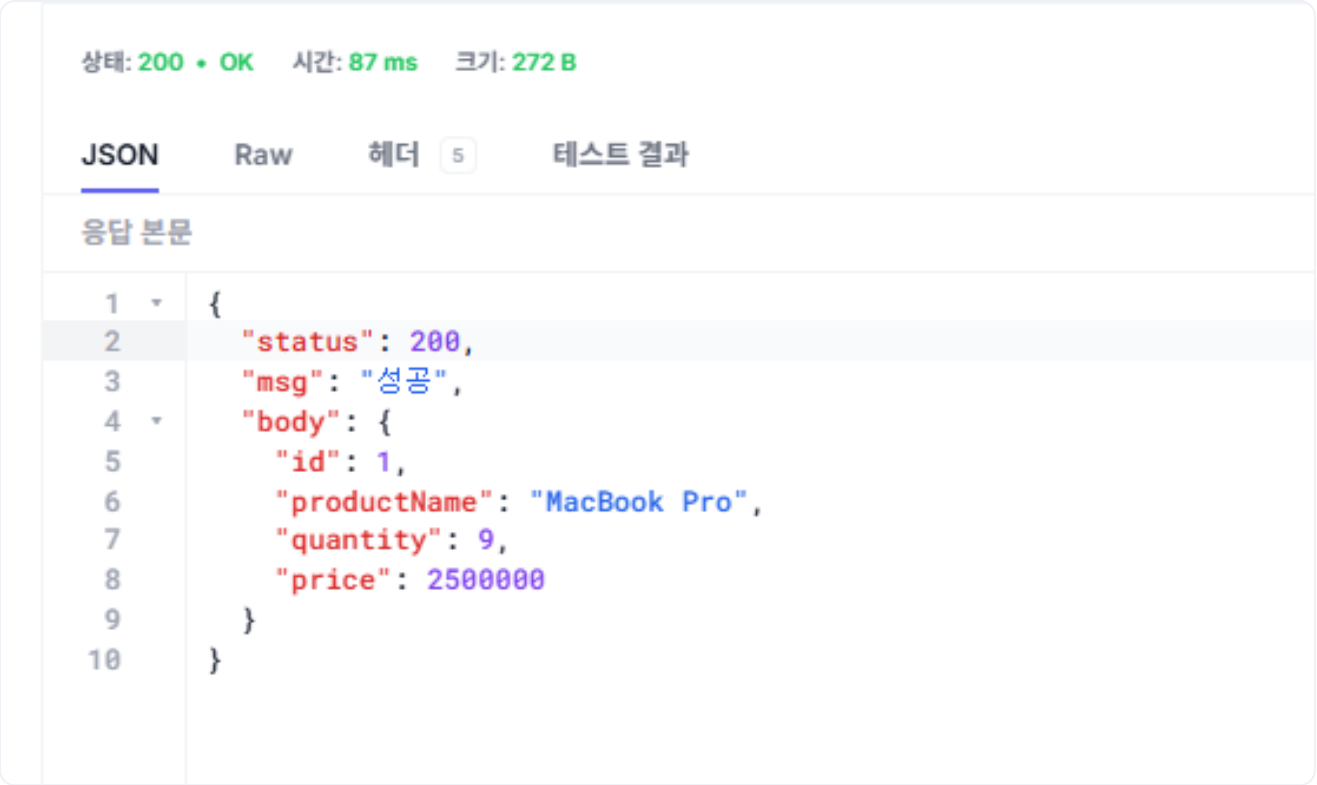


그림 2-9. 재고 감소 확인

배달 서비스에도 배달이 생성됐는지 확인합니다.

Hoppscotch **배달 조회**

```
GET http://localhost:8084/api/deliveries/4   # 더미 배달 3건 다음이라 새 배달은 4번
```

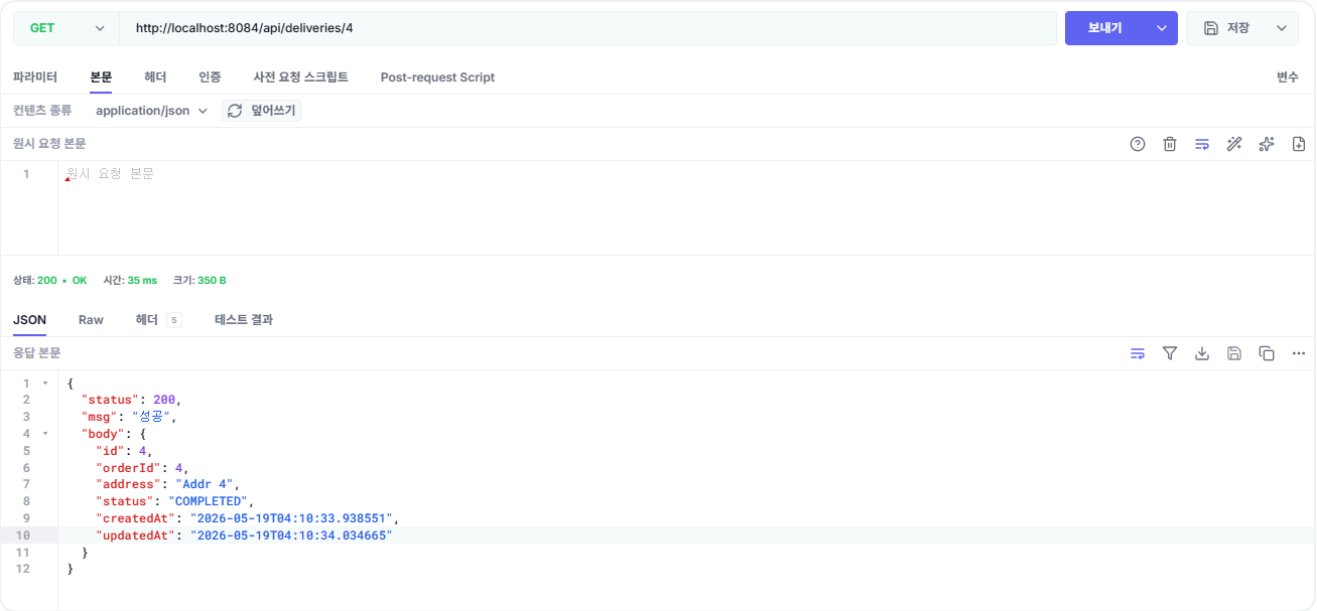


그림 2-10. 배달 생성 확인

2.6.4 시나리오 2: 재고 부족

이번에는 상품 ID가 2인 품절 상품 iPhone 15를 주문해 보겠습니다. 첫 번째 단계인 재고 차감에서 바로 실패하므로, 보상할 작업이 없어 즉시 에러가 반환됩니다.

Hoppscotch 주문 생성 (재고 부족)

```
POST http://localhost:8081/api/orders

{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 4"
}
```



그림 2-11. 재고 부족 시 에러 응답

2.6.5 시나리오 3: 주소 누락

이번에는 주소를 빈 문자열로 보내 보겠습니다. 주문 자체는 재고 차감까지 진행되지만, 배달 서비스에서 주소가 없으므로 실패합니다. 이때 보상 트랜잭션이 작동하여 차감된 재고가 복구되고, 주문 데이터는 트랜잭션 롤백으로 처음부터 없던 일처럼 DB에서 사라집니다.

Hoppscotch 주문 생성 (주소 누락)

```
POST http://localhost:8081/api/orders

{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": ""
```

```
}
```

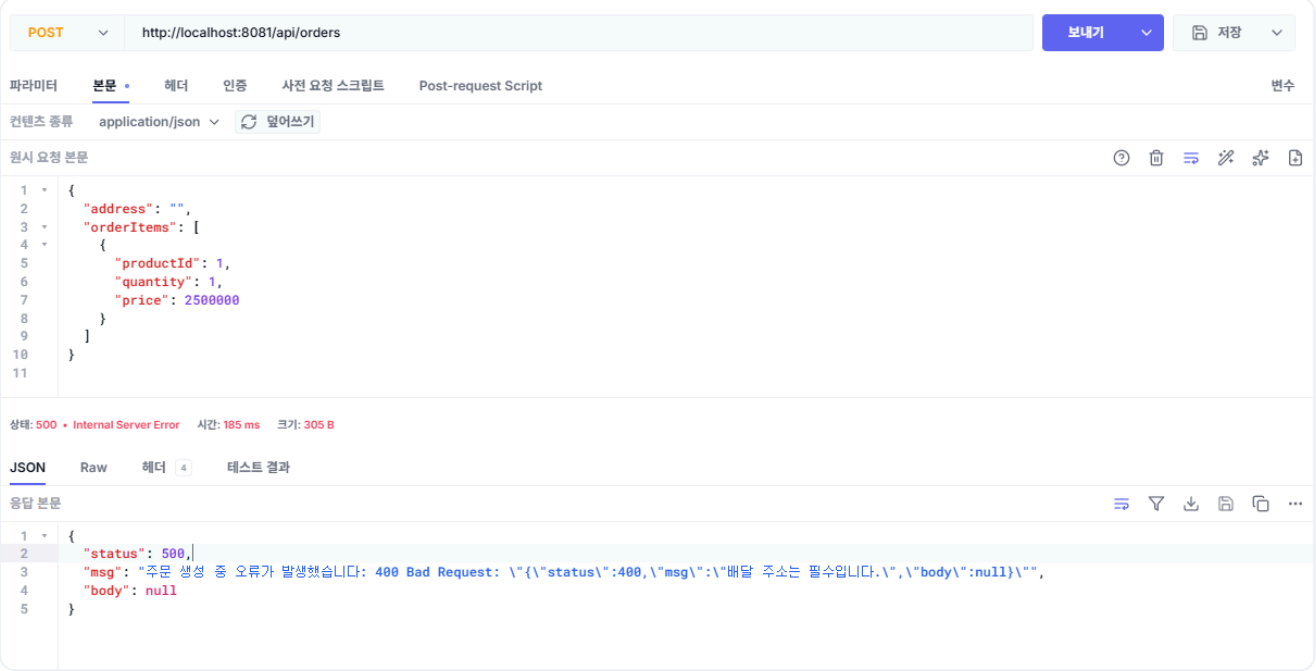


그림 2-12. 주소 누락 시 에러 응답

그리고 재고가 원복되었는지 확인합니다. 시나리오 1에서 재고가 10개에서 9개로 줄었으니, 이번에 차감된 재고가 보상으로 복구되면 다시 9개로 돌아옵니다.

Hoppscotch 재고 조회

```
GET http://localhost:8082/api/products/1
```

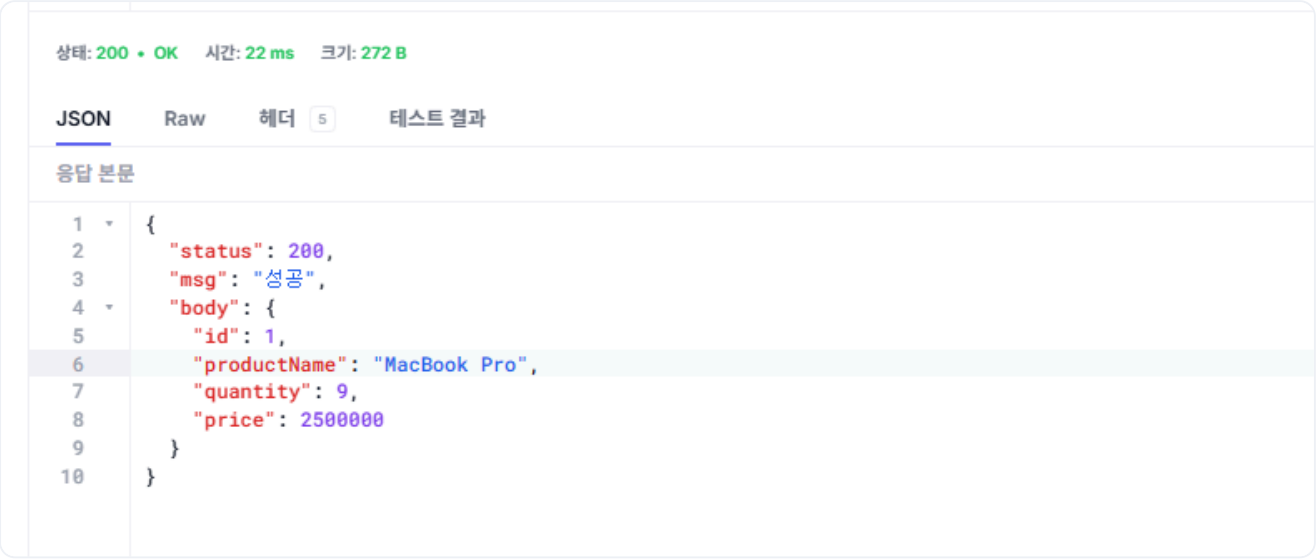


그림 2-13. 재고 원복 확인

테스트가 끝났으면 실행 중인 컨테이너를 정리합니다.

터미널 **컨테이너 정리**

```
docker compose down
```

오픈이 : "기능별로 서비스를 분리했는데도 서로 통신이 되네요. 중간에 실패해도 보상 트랜잭션이 동작하구요."

선배 : "맞아요. 그런데 이 방식은 주문 서비스가 상품이랑 배달을 직접 호출하다 보니, 서비스만 분리됐을 뿐 결국 다른 서비스의 장애가 그대로 전파돼요. 상품이나 배달 서비스가 조금만 느려지거나 죽어도, 주문 서비스까지 같이 느려지거나 멈추죠. 이제 여기서부터 하나씩 고쳐 나가봅시다."

지금 구조는 각 서비스 안에서 컨트롤러 계층과 서비스 계층이 직접 의존합니다. 이 때문에 처리 방식을 비동기로 전환하려면 컨트롤러를 비롯해 연관된 메서드까지 모두 수정해야 합니다.

다음 챕터에서는 이 결합도를 낮추기 위해 아키텍처를 개선합니다. 외부 요청과 비즈니스 로직을 분리해, 어느 한쪽을 변경하더라도 다른 쪽에 영향을 주지 않는 구조를 만듭니다.

이것만은 기억하자

- → 서비스가 분리되면 세션을 공유할 수 없어, **JWT**로 인증하고 호출할 때 토큰을 전달합니다.
- → 분산 트랜잭션은 자동으로 롤백되지 않아, 실패하면 **보상 트랜잭션**으로 이미 끝난 작업을 되돌립니다.
- → 동기 직접 호출은 서비스 간 **결합도를 높여**, 한 서비스가 멈추면 호출한 쪽도 함께 멈춥니다.